Unity Async and Awaitable Guide (Beginner-Friendly)

This guide teaches `async` / `await` from scratch, then shows how to use Unity 6 `Awaitable` in real gameplay code.

You will learn:

- What asynchronous code is in plain language.
- How `Task`, `async`, and `await` work.
- Why Unity code can fail when using generic .NET async patterns.
- Why `Awaitable` is usually the better fit for gameplay timing in Unity 6.
- How to build safe loops with cancellation.
- How to avoid the "fires once then stops" bug in weapon systems.

---

1) Async in Plain English

Synchronous code

Synchronous code runs line by line and waits for each line to finish.

Example:

```
LoadLevel();
SpawnEnemies();
PlayMusic();
```

Each call blocks the next one.

Asynchronous code

Asynchronous code lets you start work, then pause without freezing the game, then continue later.

That is useful for things like:

- wait 0.5 seconds between shots,
- wait for web requests,
- wait for save/load operations,
- stagger effects over multiple frames.

The key point: async is mostly about **not blocking** your main flow.

---

2) The Three Keywords: `Task`, `async`, `await`

`Task`

Represents work that may finish later.

`async`

Marks a method that can use `await`.

`await`

Pauses this method until awaited work completes, then resumes.

Simple example:

```
private async Task ExampleAsync() {
    await Task.Delay(500);
    Debug.Log("Half a second later");
}
```

Return type rules

- Prefer `Task` / `Task<T>` for .NET async methods.
- In Unity 6 gameplay code, prefer `Awaitable` / `Awaitable<T>` when available.
- Avoid `async void` except event handlers. It is hard to track and errors can become invisible.

---

3) Why Unity Needs Special Care

Unity APIs (like `transform`, `Instantiate`, `Destroy`, `GetComponent`) must be used from the Unity main thread.

With generic `Task.Delay`, continuation behavior can be tricky depending on context and runtime setup. If continuation runs in an unexpected context, your next Unity API call can throw and kill your loop.

Symptoms:

- first shot works,
- later shots stop,
- no obvious error in gameplay if the async task is fire-and-forget.

---

4) Unity 6 `Awaitable` Basics

Unity 6 provides `Awaitable` APIs tuned for engine workflows.

Common helpers:

- `Awaitable.WaitForSecondsAsync(seconds, token)`
- `Awaitable.NextFrameAsync(token)`

Why this is great:

- clearer intent for frame/gameplay waits,
- better fit for Unity lifecycle,
- less friction than generic timer-based delays for gameplay loops.

Example:

```
private async Awaitable FlashAsync(CancellationToken token) {
    spriteRenderer.enabled = false;
    await Awaitable.WaitForSecondsAsync(0.1f, token);
    spriteRenderer.enabled = true;
}
```

---

5) Cancellation: The Safety Belt

Long-running loops must be cancelable.

Use `CancellationTokenSource` (`CTS`) to stop async work when:

- target list becomes empty,
- object gets destroyed,
- state changes (weapon unequipped, player dead, scene changed).

Pattern:

```
private CancellationTokenSource _cts;

private void StartLoop() {
    StopLoop();
    _cts = new CancellationTokenSource();
    _ = LoopAsync(_cts.Token);
}

private void StopLoop() {
    if (_cts == null) return;
    _cts.Cancel();
    _cts.Dispose();
    _cts = null;
}
```

Do not forget `OnDestroy` :

```
private void OnDestroy() {
    StopLoop();
}
```

---

6) Fire-and-Forget Without Silent Death

When you call `_ = SomeAsyncMethod();` , no one is awaiting it. If an exception happens, your method can stop and you may not notice quickly.

Fix:

- Catch `OperationCanceledException` as expected control flow.
- Catch unexpected `Exception` and log it.
- Stop/cleanup loop on fatal error.

Pattern:

```
private async Awaitable RunAsync(CancellationToken token) {
    try {
        while (!token.IsCancellationRequested) {
            // work
            await Awaitable.NextFrameAsync(token);
```

```
        }
    }
    catch (OperationCanceledException) {
        // normal
    }
    catch (Exception ex) {
        Debug.LogException(ex, this);
        StopLoop();
    }
}
```

---

7) Common Async Bugs in Unity

Bug A: Restarting loop on every trigger enter

If each enemy entering does:

- cancel old loop,
- start new loop,

you can get unstable behavior and race-like stop/start jitter.

Fix: only start loop when count transitions from 0 to 1.

Bug B: Null target still in list

Enemy destroyed in range might leave stale `null` in list.

Fix: prune list each loop with `RemoveAll(e => e == null)`.

Bug C: Projectile prefab missing `Rigidbody2D`

`GetComponent` can return null, and next line throws.

Fix: guard with `TryGetComponent`, log warning, continue safely.

Bug D: `async void`

`async void` methods are hard to observe and test.

Fix: use `Awaitable` / `Task` return type whenever possible.

---

8) Case Study: Weapon That Shoots Once Then Stops

Typical root causes

- loop restarted too often,
- hidden exception after first shot,
- stale target reference,
- cancellation source lifecycle not managed.

Robust strategy

1. Keep exactly one loop active.
2. Prune invalid targets before targeting.

3. Handle missing components defensively.
4. Use `Awaitable.WaitForSecondsAsync` with cancellation.
5. Catch and log unexpected exceptions.

---

9) Production-Ready `WeaponHands` (Full Listing)

This version is async-based, Unity 6 friendly, and resilient.

```
using System;
using System.Collections.Generic;
using System.Threading;
using Controllers;
using Data;
using UnityEngine;

namespace Gameplay.Weapons {
    public class WeaponHands : MonoBehaviour {
        [SerializeField] private WeaponConfig _weaponConfig;
        [SerializeField] private GameObject _weaponPrefab;
        [SerializeField] private List<EnemyController> _enemies = new();
        [SerializeField] private CircleCollider2D _circleCollider2D;

        private CancellationTokenSource _attackCts;
        private float _currentAtkSpeed;
        private float _currentRange;

        private void Awake() {
            _enemies ??= new List<EnemyController>();
            UpdateWeaponStats();
        }

        private void OnDestroy() {
            StopAttackLoop();
        }

        private void OnTriggerEnter2D(Collider2D other) {
            if (!other.TryGetComponent(out EnemyController enemy)) return;

            if (!_enemies.Contains(enemy)) {
                _enemies.Add(enemy);
            }

            if (_enemies.Count == 1) {
                StopAttackLoop();
                _attackCts = new CancellationTokenSource();
                _ = AttackEnemyAsync(_attackCts.Token);
            }
        }

        private void OnTriggerExit2D(Collider2D other) {
            if (!other.TryGetComponent(out EnemyController enemy)) return;
```

```
            _enemies.Remove(enemy);

            if (_enemies.Count == 0) {
                StopAttackLoop();
            }
        }

        private void UpdateWeaponStats() {
            _currentRange = _weaponConfig.WeaponRange;
            _circleCollider2D.radius = _currentRange;
            _currentAtkSpeed = _weaponConfig.WeaponAtkSpeed;
        }

        private void StopAttackLoop() {
            if (_attackCts == null) return;
            _attackCts.Cancel();
            _attackCts.Dispose();
            _attackCts = null;
        }

        private async Awaitable AttackEnemyAsync(CancellationToken token) {
            try {
                while (!token.IsCancellationRequested) {
                    _enemies.RemoveAll(e => e == null);

                    if (_enemies.Count == 0) {
                        StopAttackLoop();
                        return;
                    }

                    EnemyController target = _enemies[0];
                    if (target == null) {
                        continue;
                    }

                    Vector2 direction = target.transform.position - transform.position;
                    float angle = Mathf.Atan2(direction.y, direction.x) * Mathf.Rad2Deg;
                    Quaternion rotation = Quaternion.Euler(0, 0, angle);

                    GameObject spawnedSpear = Instantiate(_weaponPrefab, transform.position,
rotation);

                    if (spawnedSpear.TryGetComponent(out Rigidbody2D rb)) {
                        rb.linearVelocity = direction.normalized *
_weaponConfig.WeaponSpeed;
                    }
                    else {
                        Debug.LogWarning($"{nameof(WeaponHands)} spawned projectile without
Rigidbody2D.", this);
                        Destroy(spawnedSpear);
                    }
```

```
                    await Awaitable.WaitForSecondsAsync(Mathf.Max(0.01f, _currentAtkSpeed),
token);
                }
            }
            catch (OperationCanceledException) {
                // expected when stopping loop
            }
            catch (Exception ex) {
                Debug.LogException(ex, this);
                StopAttackLoop();
            }
        }
    }
}
```

---

10) Line-by-Line Intent (Quick Walkthrough)

- `Count == 1` start condition ensures only one attack loop.
- `StopAttackLoop()` centralizes cancel+dispose to prevent token leaks.
- Null pruning before target selection avoids stale destroyed enemy references.
- `TryGetComponent` check avoids null exceptions on missing rigidbody.
- `Awaitable.WaitForSecondsAsync` ties wait to Unity-friendly async flow.
- Exception handling keeps issues visible and prevents silent dead loops.

---

11) Performance Notes (Practical)

- Async itself is not automatically faster than coroutines; choose by fit.
- For Unity gameplay timing, `Awaitable` avoids generic timer pitfalls and is easier to reason about in engine context.
- Keep per-loop allocations low:
  - reuse lists where possible,
  - avoid repeated heavy queries,
  - do constant-time checks in loop.
- If enemy count grows large, switch from `List` first-target model to nearest-target selection with throttled refresh.

---

12) Debug Checklist for Async Weapon Loops

When firing stops unexpectedly, check:

1. Is `OnTriggerEnter2D` called (enemy has collider + rigidbody setup)?
2. Does `_enemies.Count` remain > 0 while enemy is inside?
3. Any exception logged from attack loop?
4. Is projectile prefab missing `Rigidbody2D` ?
5. Is `_currentAtkSpeed` valid (> 0)?
6. Is loop getting canceled from another script/state change?

---

13) Reusable Patterns Cheat Sheet

Safe start/stop pattern

```
private void StartSafeLoop() {
    StopSafeLoop();
    _cts = new CancellationTokenSource();
    _ = LoopAsync(_cts.Token);
}

private void StopSafeLoop() {
    if (_cts == null) return;
    _cts.Cancel();
    _cts.Dispose();
    _cts = null;
}
```

Safe timed loop pattern

```
private async Awaitable LoopAsync(CancellationToken token) {
    try {
        while (!token.IsCancellationRequested) {
            // gameplay work
            await Awaitable.WaitForSecondsAsync(0.2f, token);
        }
    }
    catch (OperationCanceledException) {
    }
    catch (Exception ex) {
        Debug.LogException(ex, this);
    }
}
```

---

14) Final Mental Model

Use async in Unity like this:

- Keep gameplay waits explicit (`Awaitable`).
- Always own cancellation lifecycle.
- Expect object destruction and null references.
- Treat fire-and-forget as dangerous unless fully guarded.
- Log unexpected exceptions close to where they happen.

If you follow those five rules, async gameplay systems become predictable and easy to maintain.